Отчёт о воспроизведении: «Super-Convergence»

Smith, Topin, 2018 · LeNet / MNIST и ResNet-56 / CIFAR-10

Коротко

- **Основной эффект повторился.** На CIFAR-10 получено 91,95% за 10 000 обновлений против 90,54% за 80 000 у baseline. Циклический режим оказался лучше на 1,41 п. п. при восьмикратно меньшем числе шагов; в статье - 92,4% против пиковых 91,2%.
- **На MNIST результат такой же по направлению.** 1cycle дал 99,13 ± 0,04% против 99,02 ± 0,05% у длинного baseline. Короткий baseline с теми же 1407 шагами получил 98,63 ± 0,05%. Средние и выборочные SD рассчитаны по трём сидам.
- **Время сократилось вместе с бюджетом.** Обучение ResNet-56 заняло 12,61 мин вместо 100,85 мин. Отношение 8,00× относится к полным протоколам, а не к скорости одного шага или достижению одинаковой точности.
- **Числа статьи не совпали точно.** На CIFAR-10 проведён один запуск на режим, на MNIST - три. Реализация перенесена из Caffe в PyTorch с сохранением архитектуры, формы SGD и BN по группам; недостающий конечный LR для MNIST явно задан равным 10^{-5}.

1. Что проверяется

В статье предлагается сначала повышать learning rate до большого значения, затем снижать его. Выбраны два сравнения с полными бюджетами обучения. Нумерация рисунков и таблиц ниже относится к arXiv:1708.07120v3 [1].

№	Утверждение	Где в статье	Значение в статье
У1	ResNet-56 можно обучить за меньшее число шагов без потери точности	Рис. 1a; введение	10 000 шагов: 92,4%; baseline около 80 000: пик 91,2%
У2	На LeNet/MNIST 1cycle даёт лучшую точность за 12 эпох вместо 85	Табл. 2	99,25 ± 0,03% против 99,03 ± 0,04%

Дополнительные вопросы. В1: что даёт 1cycle на MNIST при одинаковом числе обновлений? В2: сокращается ли время до первой записанной точности 90% на CIFAR-10? Они отделяют итог длинного обучения от сравнения при равном бюджете и общей целевой метрике.

2. Как устроен метод

- **Learning rate (LR)** - множитель при обновлении весов. В простейшем SGD: $w_{t+1} = w_t - \eta_t g_t$, где g - градиент потерь, η - LR. Маленький шаг замедляет обучение, слишком большой может сделать его неустойчивым.
- **CLR** - циклический LR: сначала линейный рост, затем линейное снижение. В выбранном опыте CIFAR-10 один цикл занимает всё обучение: 0,1 → 3 → 0,1 за 10 000 обновлений.
- **1cycle** - один цикл с дополнительным снижением LR ниже начального значения в конце. На MNIST рост и спад занимают по пять эпох, заключительный участок - две эпохи. Одновременно momentum меняется в обратную сторону.
- **Momentum** сохраняет часть предыдущего обновления, а **weight decay** добавляет к градиенту штраф за величину весов. **BatchNorm (BN)** нормализует активации по статистикам группы примеров и накапливает статистики для тестирования.

Почему это может работать. Авторы объясняют эффект регуляризацией от большого LR: такая динамика мешает слишком точно подстраиваться под train set. LR нужно согласовывать с weight decay и другими источниками регуляризации; финальное снижение шага позволяет уточнить решение [1, раздел 3 и приложение A.2].

В LR range test авторы постепенно повышают LR и проверяют, где сохраняется качество. Этот тест не повторялся, диапазон не подбирался: использованы опубликованные настройки. Причинную роль большого LR эти сравнения не изолируют.

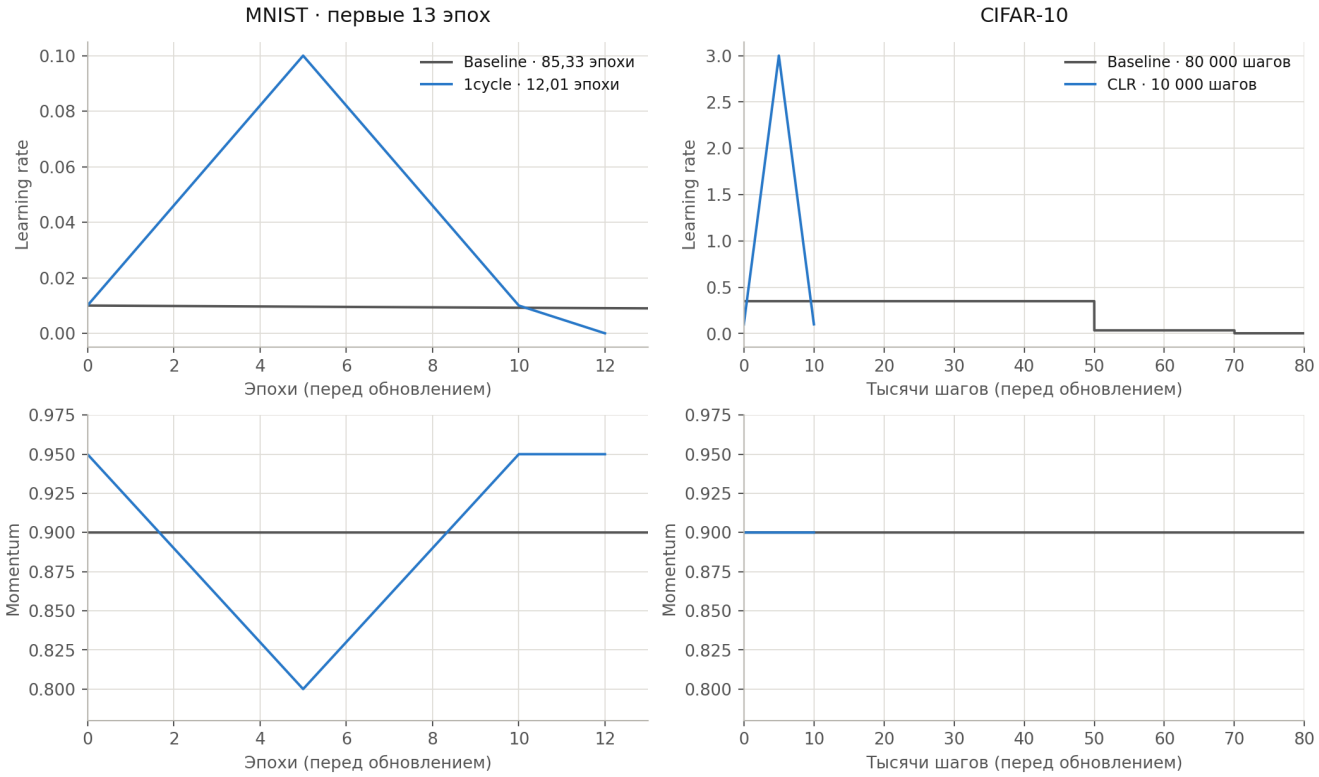


Рис. 1. Реализованные расписания LR. Для CIFAR-10 использован треугольный CLR из авторской конфигурации; для MNIST - цикл с заключительным спадом.

Шаг и эпоха. Шаг - обновление весов по батчу; эквивалент эпох = шаги × batch / размер train. Для CIFAR-10: 10 000 шагов × 1000 / 50 000 = 200 эпох.

3. Постановка

3.1. Данные, модели и расписания

Настройки CIFAR-10 сверены с архитектурой и solver-файлами репозитория авторов [2], MNIST - с таблицей 2 статьи и примером LeNet из Caffe [3]. Все обучающие и тестовые примеры сохранены; сокращения выборок и бюджетов не было.

Компонент	MNIST	CIFAR-10
Train / test	60 000 / 10 000 изображений	50 000 / 10 000 изображений
Модель	Caffe LeNet; 431 080 параметров	Авторская ResNet-56; 808 682 параметра
Batch	512	1000; BN по 8 группам × 125
Baseline	10 000 шагов ≈ 85,33 эпохи; LR $0{,}01 \cdot (1 + 10^{-4}t)^{-0{,}75}$	80 000 шагов = 1600 эпох; LR 0,35, деление на 10 после 50 000 и 70 000 шагов
Быстрый режим	1407 шагов ≈ 12,006 эпохи; LR $0{,}01 \to 0{,}1 \to 0{,}01 \to 10^{-5}$	10 000 шагов = 200 эпох; LR $0{,}1 \to 3 \to 0{,}1$; полупериод 5000 шагов
Momentum	0,9 у baseline; $0{,}95 \to 0{,}8 \to 0{,}95$ у 1cycle	0,9 у обоих режимов
Weight decay	5×10^{-4}	10^{-4}
BN moving-average fraction	BN отсутствует	0,999 у baseline; 0,95 у CLR
Входы и аугментации	Пиксели / 256; без аугментаций	RGB 0…255 минус среднее train-изображение; случайное отражение слева направо после центрирования

LeNet. Две свёртки с max pooling, полносвязный слой на 500 нейронов и выход на 10 классов. ReLU стоит только после первого полносвязного слоя; добавлять её после свёрток означало бы изменить модель Caffe.

ResNet-56. В сети 27 остаточных блоков. Переходный блок уменьшает пространственный размер, а shortcut использует average pooling. Нулевые каналы дописываются после сложения ветвей и ReLU. Сохранён авторский вариант архитектуры. Дополнительное кадрирование с padding не добавлено.

3.2. Что пришлось перенести из Caffe

В оптимизаторе Caffe LR умножает градиент до помещения в буфер momentum. Это существенно при меняющемся LR. Для одной группы параметров обновление имеет вид:

$$v_t = \mu_t v_{t-1} + \eta_t(g_t + \lambda w_t), \quad w_{t+1} = w_t - v_t.$$

Перенесены эта формула, инициализация и множители LR/weight decay для групп весов и смещений. Обычная замена на torch.optim.SGD без сверки формулы дала бы другое накопление прошлых обновлений.

3.3. Что совпадает по смыслу и что отличается

Восемь групп BN на одной GPU. В авторском запуске общий batch 1000 распределялся по восьми GPU: нормализация считалась локально по 125 изображениям. Сохранены восемь таких групп на одном устройстве, общие обучаемые веса BN и статистики первой группы для inference. Накопленные статистики нормируются как в Caffe, включая начальное накопление массы.

CPU-тесты сопоставляют сгруппированную реализацию с восемью последовательными микробатчами: выходы, градиенты и статистики совпадают с точностью вычислений. Это проверка реализации групп, а не доказательство побитового совпадения с исходным многопроцессным Caffe.

В оригинале	В воспроизведении	Что это означает
Caffe/CUDA 7, несколько GPU	PyTorch/CUDA 12.8, одна GPU; размер группы BN сохранён	Численные ядра и генераторы случайных чисел различаются
Конечный LR для хвоста MNIST не уточнён	Линейное снижение до 10^{-5}	Явное допущение; точное расписание авторов восстановить нельзя
В solver CIFAR задано 25 000 тестовых примеров с повторениями	Один проход по всем 10 000 тестовым изображениям	Иная процедура усреднения test accuracy
Табл. 5: stride 2 первой свёртки; в коде: stride 1	По рабочему коду: stride 1	Приоритет авторской реализации при расхождении источников
Несколько задач и архитектур	Два выбранных сравнения; 3 сида MNIST и 1 сид CIFAR-10	Вывод ограничен этой постановкой; разброс CIFAR не оценён

3.4. Запуски, метрики и окружение

На MNIST выполнены три режима с seeds 42, 43, 44: длинный baseline, 1cycle и короткий baseline на 1407 шагов с тем же inverse-LR, что у длинного. Внутри каждого сида начальные веса у режимов одинаковы. На CIFAR-10 выполнено по одному основному запуску с seed 42. Итого 11 обучений; ещё два пилота CIFAR по 200 шагов хранятся отдельно и в основные кривые не входят.

Метрика - accuracy на полном test set после последнего обновления, без выбора лучшего checkpoint. Промежуточные оценки CIFAR записаны каждые 100 шагов, MNIST - каждые 118 шагов и в конце. Для MNIST указаны среднее и выборочное SD между запусками (ddof=1), а не стандартная ошибка среднего.

Оборудование: NVIDIA RTX PRO 6000 Blackwell Server Edition, 94,97 GiB доступной VRAM. Python 3.13.15, PyTorch 2.10.0+cu128, torchvision 0.25.0+cu128, CUDA 12.8, cuDNN 9.10.2, драйвер 580.82.07. Все вычисления FP32; AMP и TF32 отключены. Данные обходятся последовательно с переходом к началу, без случайного перемешивания.

4. Результаты

4.1. ResNet-56 на CIFAR-10: главное сравнение

Постановка: полный train set, batch 1000, 80 000 шагов baseline и 10 000 шагов CLR. Настройки из раздела 3, один запуск каждого режима с seed 42. Основная метрика - финальная точность на 10 000 тестовых изображений.

Показатель	Baseline	CLR
Шагов / эпох	80 000 / 1600	10 000 / 200
Финальная точность	**90,54%**	**91,95%**
Лучшая записанная точность	90,63%; шаг 73 400	91,95%; шаг 10 000
Точность при 10 000 шагах	53,43%	91,95%
Финальный train / test loss	0,00037 / 0,38883	0,00057 / 0,31676

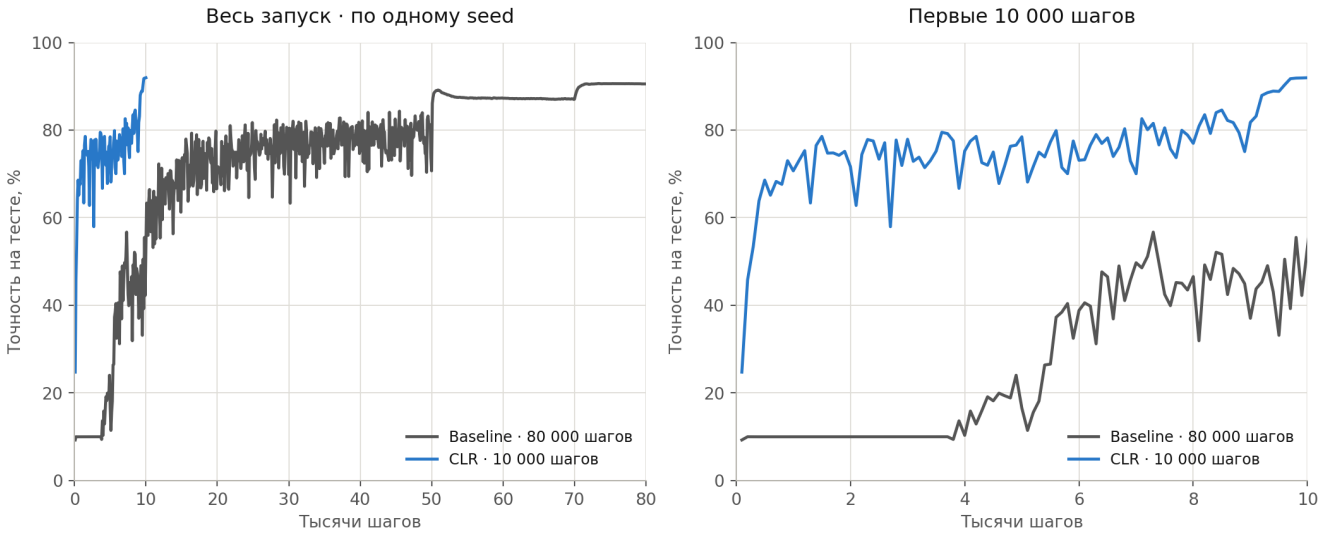


Рис. 2. Полные кривые и первые 10 000 шагов крупнее. Отрезки соединяют записанные оценки без сглаживания.

- **У1 воспроизвелось качественно.** CLR завершил обучение с преимуществом 1,41 п. п. и в восемь раз меньшим числом обновлений. При одинаковых 10 000 шагах разница составляет 38,52 п. п., но длинный baseline на этом шаге ещё не дошёл до своего первого снижения LR.
- **Точность ниже опубликованной.** Авторы указывают пик baseline 91,2%, поэтому сравнивать его нужно с полученным пиком 90,63%: разница −0,57 п. п. CLR дал 91,95% против авторских 92,4%: −0,45 п. п. Финальные 90,54% baseline не подменяют его лучший результат.
- **Разница относится к двум протоколам целиком.** Помимо LR меняется moving-average fraction BN: 0,999 против 0,95. Поэтому большой разрыв в начале кривых нельзя объяснить только расписанием шага. Подбор оптимального baseline для 10 000 обновлений и отдельные абляции BN не проводились.

Оба режима почти обнулили тренировочную потерю, но CLR имеет меньшую тестовую потерю. Это согласуется с направлением авторского объяснения, однако одного такого сравнения недостаточно, чтобы отдельно доказать механизм регуляризации большим LR.

4.2. LeNet на MNIST: длинный и короткий baseline

Постановка: batch 512, seeds 42, 43, 44. Длинный baseline содержит 10 000 обновлений (85,33 эквивалентных эпохи), 1cycle и короткий baseline - по 1407 (12,006 эпохи). Короткий baseline - дополнительный контроль B1; соответствующей отдельной строки в таблице 2 статьи нет.

Режим	Seed 42	Seed 43	Seed 44	Среднее ± SD
Baseline, 10 000	99,01	99,07	98,98	99,02 ± 0,05%
1cycle, 1407	99,14	99,16	99,08	99,13 ± 0,04%
Baseline, 1407	98,61	98,68	98,59	98,63 ± 0,05%

Все числа - проценты финальной точности. SD рассчитано по трём независимым сидам; в пределах одного сида сравнение парное по начальным весам.

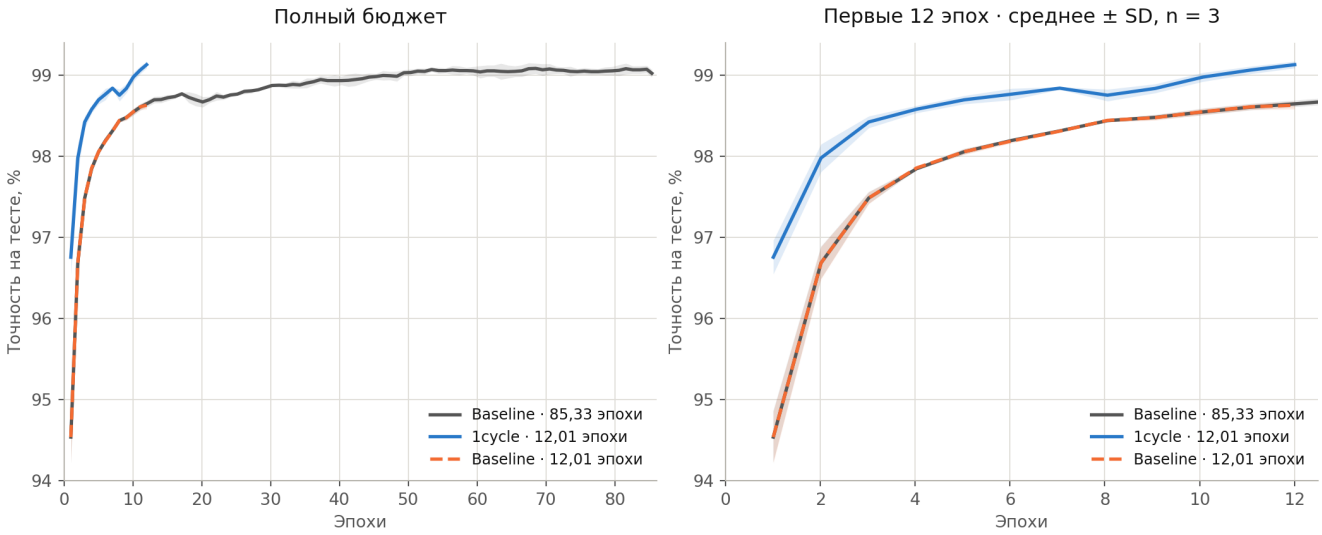


Рис. 3. Средние кривые по трём сидам и разброс между ними. Справа показан одинаковый начальный бюджет до 12 эпох.

- **У2 воспроизвелось по направлению.** 1cycle лучше длинного baseline во всех трёх парных повторах; средняя разница +0,107 п. п. При этом шагов в 7,11 раза меньше. Такой результат поддерживает полезность режима на этой задаче, но не гарантирует выигрыш для любой инициализации.
- **Ответ на B1: при равном бюджете 1cycle тоже лучше.** Преимущество над коротким baseline составляет 0,500 п. п. Это показывает, что результат не сводится к простому раннему прекращению обычного обучения. Одновременно с LR у 1cycle меняется momentum; их отдельные вклады не измерены.

Финальная точность	Статья, табл. 2	Воспроизведение	Разница средних
Baseline	99,03 ± 0,04%	99,02 ± 0,05%	−0,010 п. п.
1cycle	99,25 ± 0,03%	99,13 ± 0,04%	−0,123 п. п.

5. Сколько это стоило по времени

Постановка: та же GPU и те же запуски. Обучение и оценка измерены отдельно через perf_counter с синхронизацией CUDA на границах интервалов. Счётчик обучения не включает скачивание и загрузку данных, оценку, запись журналов и checkpoints.

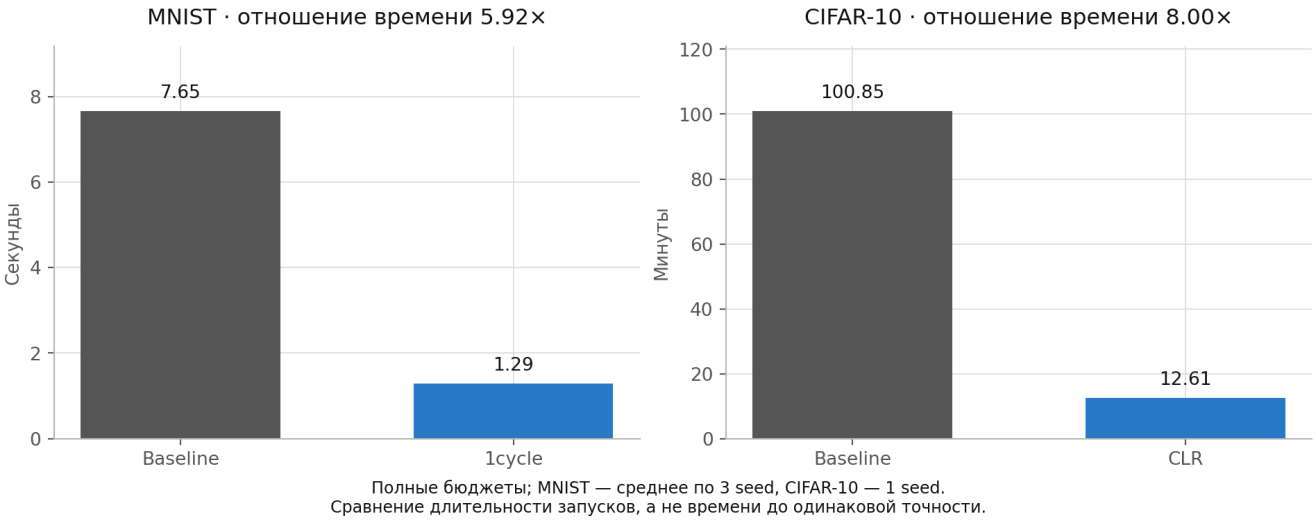


Рис. 4. Время обучения: полные бюджеты, один сид CIFAR-10, среднее трёх сидов MNIST.

Сравнение	Baseline	Циклический режим	Отношение
CIFAR-10: обучение	100,85 мин	12,61 мин	8,00×
CIFAR-10: только оценки	2,67 мин	0,36 мин	-
CIFAR-10: по UTC*	103,81 мин	13,01 мин	-
MNIST: обучение, среднее	7,65 с	1,29 с	5,92×

- **На CIFAR-10 экономия почти пропорциональна числу шагов.** Средний шаг занимает 75,6 мс у baseline и 75,7 мс у CLR. 8,00× - отношение длительностей обучения на 80 000 и 10 000 шагах; сам вычислительный шаг быстрее не стал.
- **На MNIST очень короткие запуски.** Записанные синхронизированные интервалы согласуются с журналами, но это не отдельный аппаратный бенчмарк. Времена не обязаны точно повторять отношение шагов 7,11×.

* По UTC: от старта запуска до последней оценки, без предварительной загрузки данных и установки окружения.

5.1. B2: время до общей точности 90%

Первое записанное accuracy ⩾ 90%	Baseline	CLR
Шаг; время обучения	70 900; 89,38 мин	9600; 12,11 мин

Ответ на B2: да, отношение времён 7,38×. Это первые наблюдаемые пересечения, без интерполяции. Проверки шли каждые 100 шагов; точный момент неизвестен. Более ранние краткие пересечения могли остаться незаписанными.

6. Что воспроизвелось, а что нет

№	Проверка	Итог	Основание
У1	ResNet-56: лучшее качество за меньшее число обновлений	**Да, качественно**	91,95% за 10 000 шагов против финальных 90,54% за 80 000; один сид
У2	LeNet: 1cycle лучше длинного baseline	**Да, в трёх повторах**	99,13 ± 0,04% против 99,02 ± 0,05%; шагов в 7,11 раза меньше
В1	MNIST при одинаковых 1407 шагах	**1cycle лучше**	99,13% против 98,63%; +0,500 п. п. по средним
В2	CIFAR: время до первой записанной точности 90%	**CLR быстрее**	12,11 против 89,38 мин обучения; отношение 7,38×

Главный практический вывод для выбранных задач повторился: циклические режимы дали более высокую финальную точность при существенно меньшем числе обновлений.

6.1. Что можно проверить по приложенным результатам

В каталоге results/ сохранены конфигурации, CSV, текстовые журналы и итоговые сводки; исходный код находится в hw2/. Проверка подтвердила полные сетки шагов, соответствие LR и momentum расписаниям, совпадение summary с последней строкой CSV и хешей четырёх файлов обучения с переданными исходниками.

Ссылки

1. Smith L. N., Topin N. *Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates*. arXiv:1708.07120v3, 17 мая 2018. arxiv.org/pdf/1708.07120

2. Авторские архитектуры и конфигурации: github.com/lnsmith54/super-convergence. Использованный commit: ecc561594b39442661e7e0a39d2820b0f4147324.

3. Caffe: пример MNIST/LeNet, SGD и BatchNorm. github.com/BVLC/caffe.

4. PyTorch 2.10, ограничения воспроизводимости между платформами. docs.pytorch.org/docs/2.10/notes/randomness.html.

5. Запуски от 10.10.2026: results/*/config.json, metrics.csv, training.log и summary.json. Сводные расчёты - results/analysis.json и results/summary.csv.